

ESP32-S3 项目开发说明

AI 联网对话 | 无线手柄

基于 ESP32-S3 的智能交互系统

硬件平台：ESP32-S3

开发环境：PlatformIO / Arduino IDE

预计周期：15 天

文档版本：v1.0

编制日期：2026 年 5 月

目录

第一章	项目概述	3
1.1	项目背景	3
1.2	项目目标	3
1.3	系统架构	3
1.4	硬件清单	4
第二章	阶段任务与达标要求	5
2.1	阶段一：环境搭建与基础外设驱动	6
2.2	阶段二：Wi-Fi 联网与 API 通信	7
2.3	阶段三：语音采集（硅麦驱动）	8
2.4	阶段四：语音播放（扬声器驱动）	10
2.5	阶段五：云端 ASR + LLM + TTS 集成	11
2.6	阶段六：屏幕显示	13
2.7	阶段七：无线手柄通信	14
2.8	阶段八：系统联调与测试	15
第三章	总体时间线	16
第四章	关键难点与风险提示	17
4.1	技术难点	17
4.2	风险与应对	17
第五章	附录	18
5.1	推荐参考资源	18
5.2	硬件接线参考	18

项目概述

1.1 项目背景

本项目基于 ESP32-S3 微控制器，旨在开发一套具备 AI 联网对话能力和无线手柄控制功能的智能交互系统。ESP32-S3 以其强大的双核处理器、丰富的外设接口（I2S、SPI、I2C、UART 等）以及内置 Wi-Fi 和 BLE 功能，非常适合作为本项目的主控芯片。

1.2 项目目标

- AI 联网对话：**通过硅麦（PDM 数字麦克风）采集语音，上传至云端进行语音识别（ASR），调用大语言模型（LLM）API 生成回复，再通过 TTS 语音合成后经扬声器播放，实现完整的语音对话闭环。
- 无线手柄：**基于 Wi-Fi（UDP/TCP）通信，实现外接手柄与 ESP32-S3 之间的无线数据传输，用于控制 PWM 显示器等外设。
- 屏幕显示：**在 OLED 或 LCD 屏幕上实时显示对话内容、手柄状态等信息。

1.3 系统架构

1.4 硬件清单

序号	器件	备注
1	ESP32-S3 开发板	主控芯片，推荐 ESP32-S3-DevKitC 或合宙等型号
2	PDM 硅麦克风	如 INMP441、MSM261S4030H0 等，用于语音采集
3	I2S 扬声器/功放模块	MAX98357 + 小喇叭，用于语音播放
4	OLED/LCD 显示屏	0.96 寸 SSD1306 OLED 或 1.8 寸 TFT
5	按钮/开关	按键触发录音、模式切换等
6	无线手柄 (ESP32)	第二块 ESP32 作为手柄端，发送控制指令
7	PWM 显示器	已有的外设，通过手柄控制

阶段任务与达标要求

阶段	任务内容	达标要求	预计天数
一	环境搭建与基础外设驱动	见 §2.1	2 天
二	Wi-Fi 联网与 API 通信	见 §2.2	2 天
三	语音采集（硅麦驱动）	见 §2.3	2 天
四	语音播放（扬声器驱动）	见 §2.4	1 天
五	云端 ASR + LLM + TTS 集成	见 §2.5	3 天
六	屏幕显示	见 §2.6	1 天
七	无线手柄通信	见 §2.7	2 天
八	系统联调与测试	见 §2.8	2 天

说明：表格中天数为预估上限，实际进度以达标要求为准。

2.1 阶段一：环境搭建与基础外设驱动

预估：2 天 优先级：★★★★★

子任务清单

1.1 开发环境搭建

- 安装 VS Code + PlatformIO 插件，或配置 Arduino IDE
- 创建项目工程，配置 ESP32-S3 板级支持包
- 配置串口调试工具（如 Putty 或 Arduino 串口监视器）

1.2 基础外设驱动

- 点亮板载 LED，实现 Blink 例程
- 驱动 OLED/LCD 屏幕，显示”Hello World”
- 测试按钮中断输入

1.3 FreeRTOS 任务框架搭建

- 搭建多任务框架（语音采集任务、AI 处理任务、显示任务、手柄处理任务）
- 实现任务间通信（队列/信号量）

达标要求

编译、烧录、串口输出均正常

屏幕能正常显示自定义文字

能通过按钮触发中断并在串口输出日志

多任务能同时运行且互不干扰

2.2 阶段二：Wi-Fi 联网与 API 通信

预估：2 天 优先级：★★★★★

子任务清单

2.1 Wi-Fi 联网

- 编写 Wi-Fi 连接代码（Station 模式）
- 实现自动重连机制
- 处理断网异常情况

2.2 HTTP/HTTPS 请求

- 使用 `HTTPClient` 库发送 GET/POST 请求
- 测试访问公共 API（如天气 API），验证联网通畅
- 处理 JSON 响应（使用 `ArduinoJson` 库）

2.3 大模型 API 对接准备

- 注册并获取 API Key（如讯飞星火、百度文心、阿里通义千问等）
- 阅读 API 文档，确认请求格式和鉴权方式
- 实现 HTTP POST 发送对话请求并接收回复

达标要求

ESP32 能成功连接指定 Wi-Fi

能发送 HTTP 请求并正确解析 JSON 响应

能调用大模型 API 并获取文本回复（可先用串口打印验证）

2.3 阶段三：语音采集（硅麦驱动）

预估：2 天 优先级：★★★★★

这是当前任务的**核心难点**，需要重点关注。

子任务清单

3.1 硬件连接

- 确认硅麦引脚定义（DATA、CLK、L/R、VDD、GND）
- 连接至 ESP32-S3 的 I2S 接口
- 注意：ESP32-S3 的 I2S 引脚与经典 ESP32 有所不同，务必查阅引脚定义表

3.2 I2S 驱动配置

- 使用 ESP-IDF 的 I2S 驱动库或 Arduino 的 I2SSampler 库
- 配置采样率（建议 16kHz）、位深（16bit）、通道数（单声道）
- 配置 PDM 模式（硅麦通常是 PDM 输出）

3.3 音频数据读取

- 从 I2S 接口读取 PDM 音频数据
- 将 PDM 数据转换为 PCM 数据（软件解码）
- 将音频数据缓存到内存中（使用环形缓冲区）

3.4 录音控制逻辑

- 实现按键触发录音（按下开始、松开停止）
- 或实现 VAD（语音活动检测），自动检测说话开始和结束
- 录音完成后生成 WAV 文件头（或直接裸音频数据），准备上传

达标要求

能通过硅麦采集到音频数据，串口打印出数值（有声音时数据明显变化）

能录制一段语音并保存为 WAV 格式（或裸 PCM 数据）

录制音频能通过 PC 播放验证（导出到电脑后可听）

参考代码结构

```
// I2S 配置 (PDM 模式)
i2s_config_t i2s_config = {
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX | I2S_MODE_PDM),
    .sample_rate = 16000,
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
    .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
    .communication_format = I2S_COMM_FORMAT_STAND_I2S,
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
    .dma_buf_count = 8,
    .dma_buf_len = 256,
};
```

2.4 阶段四：语音播放（扬声器驱动）

预估：1 天 优先级：★★★★

子任务清单

4.1 I2S 扬声器驱动

- 连接 MAX98357（或类似 I2S 功放模块）至 ESP32-S3
- 配置 I2S 为 TX（输出）模式
- 播放预置的音频数据（如简单的提示音）

4.2 TTS 音频播放

- 接收云端返回的 TTS 音频数据（MP3 或 PCM）
- 若为 MP3，需使用 ESP8266Audio 或类似库解码
- 通过 I2S 扬声器播放

达标要求

扬声器能正常播放提示音或音频文件

能播放云端返回的 TTS 语音回复

音质清晰可辨

2.5 阶段五：云端 ASR + LLM + TTS 集成

预估：3 天 优先级：★★★★★

这是 AI 对话的核心逻辑链，需要把三个阶段二-四打通。

子任务清单

5.1 云端服务选型

- 确定语音识别（ASR）服务：如讯飞语音听写、百度语音识别、阿里语音识别
- 确定大语言模型（LLM）服务：如讯飞星火、百度文心一言、阿里通义千问、DeepSeek
- 确定语音合成（TTS）服务：如讯飞 TTS、百度 TTS、Edge TTS
- 推荐选型方案见下表

5.2 ASR 集成

- 将阶段三录制的音频数据通过 HTTP POST 发送至 ASR 接口
- 解析返回的识别文本
- 处理好超时和错误情况

5.3 LLM 集成

- 将 ASR 识别文本作为 Prompt 发送至大模型 API
- 设置系统提示词（System Prompt），定义 AI 角色
- 解析返回的回复文本

5.4 TTS 集成

- 将 LLM 回复文本发送至 TTS 接口
- 接收返回的音频数据（MP3 或 PCM）
- 传递给阶段四的播放模块

5.5 完整对话流程串联

- 按下按键 → 录音 → ASR → LLM → TTS → 播放 → 等待下一次输入
- 在串口和屏幕上显示每个环节的状态
- 加入超时保护和错误重试机制

云端服务推荐方案

环节	推荐服务	优势	注意
ASR	讯飞语音听写	识别准确率高,支持中文	需要申请 AppID
ASR	百度语音识别	免费额度多	文档完善
LLM	DeepSeek API	性价比高, 响应快	国内可直连
LLM	阿里通义千问	免费额度充足	兼容 OpenAI 格式
TTS	Edge TTS (免费)	免费, 语音自然	需要 HTTP 请求模拟
TTS	讯飞 TTS	多音色可选	需要付费

达标要求

按住按钮说话 → 松开后 → 屏幕显示识别文字 → 显示 AI 回复 → 扬声器播放语音

整个闭环能在 5 秒内完成（取决于网络和 API 速度）

能稳定连续对话 3 轮以上不出错

2.6 阶段六：屏幕显示

预估：1 天 优先级：★★★

子任务清单

6.1 屏幕 UI 设计

- 设计对话界面：显示”识别中…””AI 回复：XXX”等
- 设计手柄状态界面：显示连接状态、PWM 值等
- 实现多页切换（对话页 / 手柄状态页）

6.2 中文显示

- 生成中文字库（取模工具如 PCtoLCD2002）
- 实现 UTF-8 到 GB2312 的编码转换（如有必要）
- 确保屏幕能正确渲染中文对话内容

6.3 滚动显示

- 当文字超出一屏时，实现自动滚动或翻页
- 显示对话历史记录

达标要求

屏幕能正确显示中文对话内容

能显示手柄连接状态和 PWM 数值

文字超长时能滚动/翻页显示完全

2.7 阶段七：无线手柄通信

预估：2 天 优先级：★★★

子任务清单

7.1 手柄端（发射端）开发

- 使用第二块 ESP32（或 ESP8266）作为手柄
- 读取摇杆/按键的模拟值和数字值
- 通过 Wi-Fi UDP 发送控制数据包

7.2 接收端开发

- ESP32-S3 作为 UDP 服务器，监听手柄数据
- 解析数据包，提取控制指令
- 将指令转发至 PWM 显示器

7.3 通信协议设计

- 定义数据包格式（建议使用结构化数据）

数据包格式参考

// 手柄数据包结构（建议 6 字节）

Byte 0	Byte 1	Byte 2-3	Byte 4-5
-----	-----	-----	-----
帧头 0xAA	指令类型	PWM 通道 1	PWM 通道 2

指令类型：

- 0x01 → 设置 PWM 值
- 0x02 → 读取状态
- 0x03 → 心跳包

达标要求

手柄端能发送控制指令，接收端能正确接收

通信延迟 <100ms（同一局域网内）

手柄能稳定控制 PWM 显示器输出

2.8 阶段八：系统联调与测试

预估：2 天 优先级：★★★

子任务清单

8.1 全功能联调

- AI 对话 + 手柄控制同时运行
- 测试任务间资源竞争和优先级问题
- 优化内存使用（ESP32-S3 的 PSRAM 分配）

8.2 异常处理

- Wi-Fi 断线自动重连
- API 调用超时处理
- 录音失败/播放失败容错

8.3 性能优化

- 降低端到端对话延迟
- 优化音频缓冲区大小，减少卡顿
- 检查内存泄漏

8.4 文档整理

- 整理完整代码结构和注释
- 编写 README 说明文档（硬件接线图、API 配置方法）
- 整理 BOM 物料清单

达标要求

AI 对话和手柄控制能同时稳定运行

断网后能自动恢复连接

连续运行 30 分钟无崩溃

文档完整，后续人员能按文档复现

总体时间线

天次	阶段	优先级	任务
第 1-2 天	一	最高	环境搭建、基础外设驱动
第 3-4 天	二	最高	Wi-Fi 联网、API 通信
第 5-6 天	三	最高	硅麦驱动与语音采集
第 7 天	四	高	I2S 扬声器驱动
第 8-10 天	五	最高	ASR + LLM + TTS 集成
第 11 天	六	中	屏幕显示 UI
第 12-13 天	七	高	无线手柄通信
第 14-15 天	八	高	联调、测试、文档

备注：阶段三（语音采集）和阶段五（AI 集成）是此项目的核心难点，建议优先投入精力攻克。无线手柄部分可在 AI 对话功能稳定后再开展。

关键难点与风险提示

4.1 技术难点

- 1. **PDM 麦克风驱动**——ESP32-S3 的 I2S 在 PDM 模式下配置较为复杂,DMA 缓冲区大小和采样率需要反复调试。建议先用 Arduino 的 I2S 库跑通基础采集,再考虑性能优化。
- 2. **音频数据上传**——WAV 文件较大 ($16\text{kHz} \times 16\text{bit} \approx 32\text{KB/s}$), ESP32 内存有限,需合理分片上传或流式上传 (Chunked Transfer)。
- 3. **端到端延迟**——录音 $\rightarrow$ ASR $\rightarrow$ LLM $\rightarrow$ TTS $\rightarrow$ 播放,每个环节都会引入延迟。整体控制在 3-5 秒内为佳。
- 4. **内存管理**——ESP32-S3 虽然有 PSRAM,但多个任务同时运行 (Wi-Fi、I2S、显示、手柄)时要注意栈空间分配。

4.2 风险与应对

风险	影响	应对措施
ASR 识别率低	对话体验差	先测试不同 ASR 服务,选用最合适的
API 请求超时	对话中断	加入超时重试机制,设置合理超时阈值
硅麦采集失败	无法输入语音	先用手头的 USB 麦克风在 PC 端做 ASR 测试,验证云端链路再调试硬件
Wi-Fi 不稳定	对话/手柄中断	加入自动重连,界面显示连接状态
扬声器音量小	听不清回复	检查功放模块供电和增益设置

附录

5.1 推荐参考资源

- ESP32-S3 技术参考手册（乐鑫官方）
- Arduino-ESP32 I2S 文档:<https://docs.espressif.com/projects/arduino-esp32/>
- PlatformIO 官方文档: <https://platformio.org/>
- 讯飞开放平台语音识别 API 文档
- DeepSeek API 文档: <https://platform.deepseek.com/>
- Edge TTS 非官方 API: <https://github.com/rany2/edge-tts>

5.2 硬件接线参考

硅麦（PDM）→ ESP32-S3

硅麦引脚	说明	ESP32-S3 引脚（示例）
DATA	PDM 数据输出	GPIO 4 (I2S_IN_D0)
CLK	PDM 时钟输入	GPIO 5 (I2S_BCK)
VDD	电源正极	3.3V
GND	电源地	GND
L/R	声道选择	GND（左声道）/ 3.3V（右声道）

I2S 功放 → ESP32-S3

MAX98357 引脚	说明	ESP32-S3 引脚（示例）
BCLK	I2S 位时钟	GPIO 6
LRC	左右声道时钟	GPIO 7
DIN	I2S 数据输入	GPIO 8
VIN	电源正极	5V
GND	电源地	GND

OLED 屏幕 → ESP32-S3

OLED 引脚	说明	ESP32-S3 引脚（示例）
SDA	I2C 数据	GPIO 1
SCL	I2C 时钟	GPIO 2
VCC	电源正极	3.3V
GND	电源地	GND

— 文档结束 —